# Agents и LLM: в чем разница и как их использовать в реальных проектах

# Я - Маяцкая Екатерина

-Data Scientist в Сбербанк – направление NLP
-Преподаватель в Уральском Федеральном Университете
-Заказная разработка проектов с внедрением ИИ
-Участие в международных конференциях, публикации с индексацией в Scopus

# Мой путь

0. Самостоятельное изучение Python (2020-2022)
1. Магистратура в УрФУ (2022-2024)
   - Проектный практикум (проекты от компаний)
   - Научная деятельность (участие в конференциях)
     - Выступление на публике
     - Глубокое изучение теории
     - Объяснение сложного простыми словами

2. Проекты совместно с Лабораторией ИИ (УрФУ + Сбер) (2024-2025)
   - Работа над реальными проектами
   - Работа в команде + общение с заказчиком

+ Научная деятельность

3. Работа над заказными проектами (2025)
4. Преподавание в онлайн университете Zerocoder (2024-2025)

5. Трудоустройство в Сбербанк (2025 - настоящее время)
+ Работа над заказными проектами
+Преподавание (УрФУ и МФТИ)

# Подробнее про проекты и обучение

2. Проекты совместно с Лабораторией ИИ (УрФУ + Сбер) (2024-2025)

*Изучаю подходы работы с LLM (обращаю внимание на скорость и качество, ведение общего проекта с командой)*

-Разработка сервиса генерации тестовых заданий для студентов
-Разработка валидатора текстовых документов для Архива Челябинской области
-Генерация отчетности по прогнозу погоды для МЧС Свердловской области

3. Работа над заказными проектами (2025)
Проект по созданию ИИ-помощника для сотрудников маркетплейсов (Ozon, WB, Яндекс Маркет).
Система: RAG + Локальная языковая модель

*Проблема: Сложность разбиения данных для RAG*
*Мое решение: Разбиение по смысловым разделам (LLM + моя валидация) + хранение в метаданных ссылок на оригиналы документации*

# Подробнее про проекты и обучение

4. Преподавание в онлайн университете Zerocoder (2024-2025)
Преподавание курса Промпт-Инжиниринг

Особенность: Добавила более глубокое изучение темы NLP.
Архитектура LLM, дообучение, RAG, агенты

*Для себя: углубленное изучение теории*

5. Трудоустройство в Сбербанк (2025 - настоящее время)
Основной проект: Создание сервиса для помощи Бизнес заказчикам и Аналитикам для написания БТ-ТЗ

Разработка системы для извлечения сущностей из текстов документов

*Что стараюсь внедрять:*
*Коммуникация с людьми, проведение Демо сервиса*
***Важно: Генерация идей решения проблемы = насмотренность, опыт***

# Подробнее про проекты и обучение

5. Трудоустройство в Сбербанк (2025 - настоящее время)

*Особенности работы:*

- Работа с ограниченным набором моделей: Gigachat (безлимитный доступ), DeepSeek 1 поток
- Более позднее обновление библиотек, не все библиотеки в контуре банка
- Существуют рекомендуемые решения (инструменты), которые лучше использовать в разработке
- Доступность всей необходимой инфраструктуры ( возможность получения помощи от администраторов инструментов)
- Работа в полном цикле разработки: от идеи до деплоя

*Помимо этого:*

- ДМС
- Комфортный офис с зонами отдыха
- Активности для сотрудников

# Подробнее про проекты и обучение

+ Работа над заказными проектам

Пригласили как преподавателя в Zerocoder

Два проекта:

1. Создание агента, позволяющего получить подсказки по нормативной документации для строительных организаций.
Итоговая реализация - агент с пятью инструментами, которые позволяют получить информацию из разных источников в зависимости от запроса пользователя + учтена память диалога.

2. Создание LangGraph цепочки с прерыванием, включающую в себя агента для итеративной проверки и заполнения xlsx таблицы.
Таблица содержит информацию по деталям с их характеристиками. Агент позволяет заполнять недостающую информацию, проверять заполненные поля и записывать новые данные в БД.

# Языковая модель (LLM)

**Языковая модель (LLM - Large Language Model)** — это искусственный интеллект, обученный на огромном количестве текстов. Она предсказывает следующее слово в тексте, исходя из контекста.

**Пример:**
*Если мы напишем: “В 1812 году Россия…”, модель “догадается”, что речь, скорее всего, о войне с Наполеоном, и продолжит текст.*

**Функционал языковой модели:**
- Генерация текста по запросу.
- Переводы и пересказы.
- Ответы на вопросы (в пределах её знаний).
- Структурирование информации.
- Анализ текста (тон, смысл, стиль и т.п.).

**Ограничения:**
- Модель не “знает” факты — она просто статистически предсказывает текст.
- Не имеет памяти между сессиями.
- Не умеет сама действовать — без инструкций ничего не сделает.

# Агент (Agent)

Агент — это надстройка над языковой моделью, которая не просто отвечает на запросы, а выполняет действия на их основе.

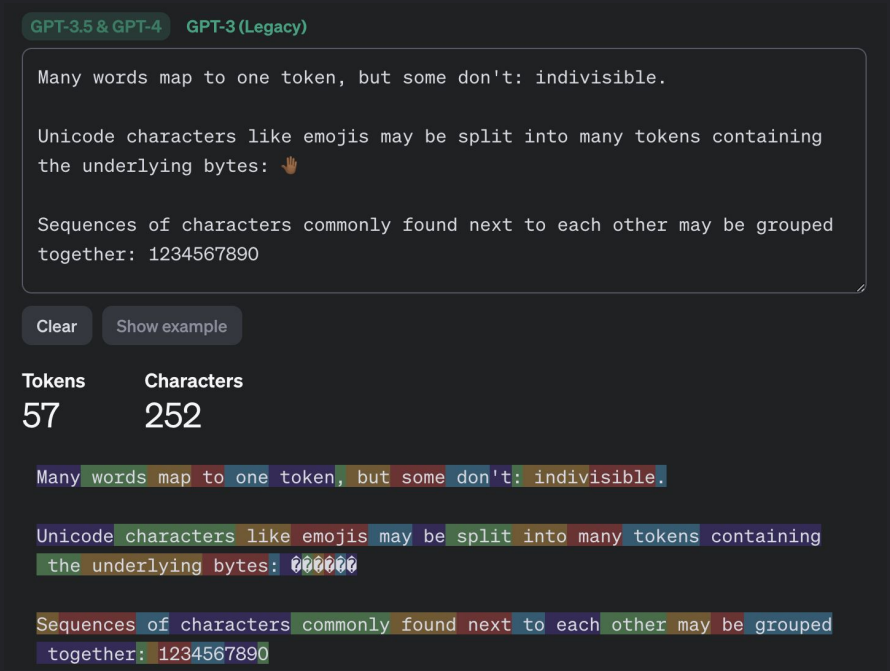
**Главная идея:**
Языковая модель → понимает запрос и генерирует текст.
Агент → ставит цели, принимает решения и вызывает инструменты, чтобы их достичь

Например, Агент в зависимости от запроса пользователя принимает решение, какое выполнить действие.

Примеры функционала:
🔍 Доступ к внешним API (погода, новости, Google, LMS, и т.д.)
📂 Анализ загруженных файлов (PDF, Word, Excel, изображения)
📅 Интеграция с календарём (создание и изменение событий)
📧 Работа с почтой (чтение, сортировка, отправка писем)
☁️ Доступ к облачным хранилищам (Google Drive, Dropbox)
📃 Планирование действий — может разбивать задачу на шаги и выполнять их последовательно
🔁 Автоматизация процессов — выполнение повторяющихся задач без участия пользователя

# Что такое токен?

**Токен** — это минимальная единица текста, с которой работает языковая модель.



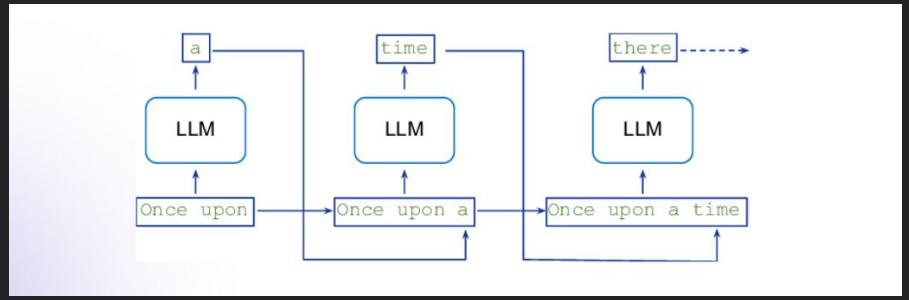

Токен – это слово, часть слова, символ или знак препинания.

# Как модель генерирует ответ?

В современных LLM (GPT, LLaMA и др.) используется BPE (Byte Pair Encoding) или её варианты.

Часто встречающиеся последовательности символов превращаются в токены.

**Модель предсказывает следующий токен!**




## Разница в токенизации разных языков

| Язык | Особенности токенизации | Среднее число токенов на слово |
|---|---|---|
| Английский | Короткие слова, частые совпадения с токенами | ~1.2 |
| Русский | Сложные, длинные слова, много морфемных вариантов | ~2–3 |
| Немецкий | Составные слова часто дробятся | 2–3 |

# На что влияет количество токенов?

- **Модель не знает фактов** → она предсказывает следующий токен на основе вероятностей.

- **Токены** — это куски текста (слова, части слов, иногда символы).
  - «ChatGPT» → токены: ["Chat", "G", "PT"].
  - В разных языках токены делятся по-разному.

- Ограничения:
  - **Длина контекста**. Если текст длинный, часть может забываться. **Стоимость**: чем больше токенов, тем дольше и дороже запрос.

- Вопрос: *«Что будет, если я дам модели текст больше, чем её контекст? Есть ли у модели ограничения»*

| TIER | RPM | TPM | BATCH QUEUE LIMIT |
|---|---|---|---|
| Free | | Not supported | |
| Tier 1 | 500 | 500,000 | 5,000,000 |
| Tier 2 | 5,000 | 2,000,000 | 20,000,000 |

https://platform.openai.com/docs/models

# Примеры функционала

## 🌐 Работа с внешними API

- Использует API погоды, новостей, Википедии, YouTube, Google Books — чтобы получать свежие данные.
- Подключается к системам электронного обучения (LMS) — например, Moodle, чтобы создавать курсы, задания, тесты.
- Работает с переводческими или голосовыми API — автоматически озвучивает текст или переводит материал.

## 📅 Интеграция с сервисами пользователя

- Доступ к календарю (Google Calendar, Outlook) — создаёт, перемещает и напоминает о событиях.
- Управление облачным хранилищем (Google Drive) — загружает и извлекает нужные файлы.

## 📚 Работа с материалами и документами

- Анализирует загруженные файлы (PDF, DOCX, таблицы) и извлекает ключевые данные.
- Создаёт резюме, отчёты, конспекты на основе загруженных документов.
- Может искать нужные материалы в коллекции

# Отличие на примере составления расписания

## Языковая модель (LLM)

Запрос: “Построй прогноз по медиапотреблению населения на следующий квартал.”
Ответ языковой модели:
“Извините, у меня нет доступа к данным о медиапотреблении.Но я могу предложить шаблон модели прогноза или помочь описать логику анализа трендов.”

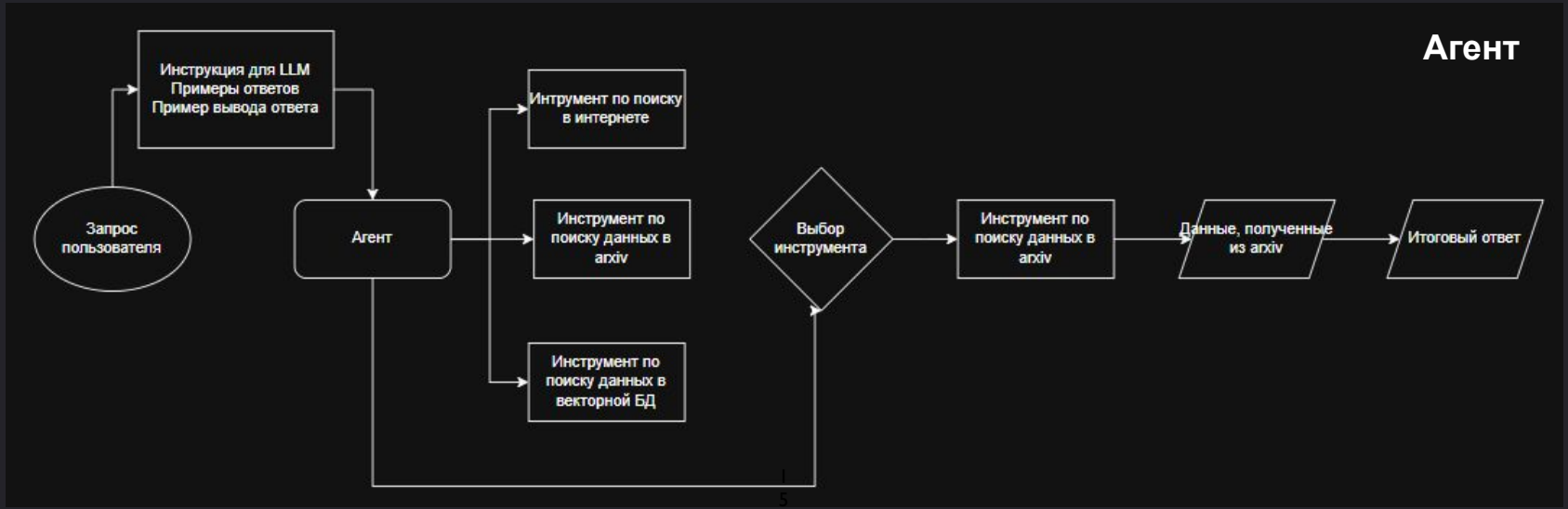
Что происходит:
* Модель понимает смысл запроса,
* генерирует ответ,
* но не может выполнить действие (она не знает, где хранится расписание).

## Агент

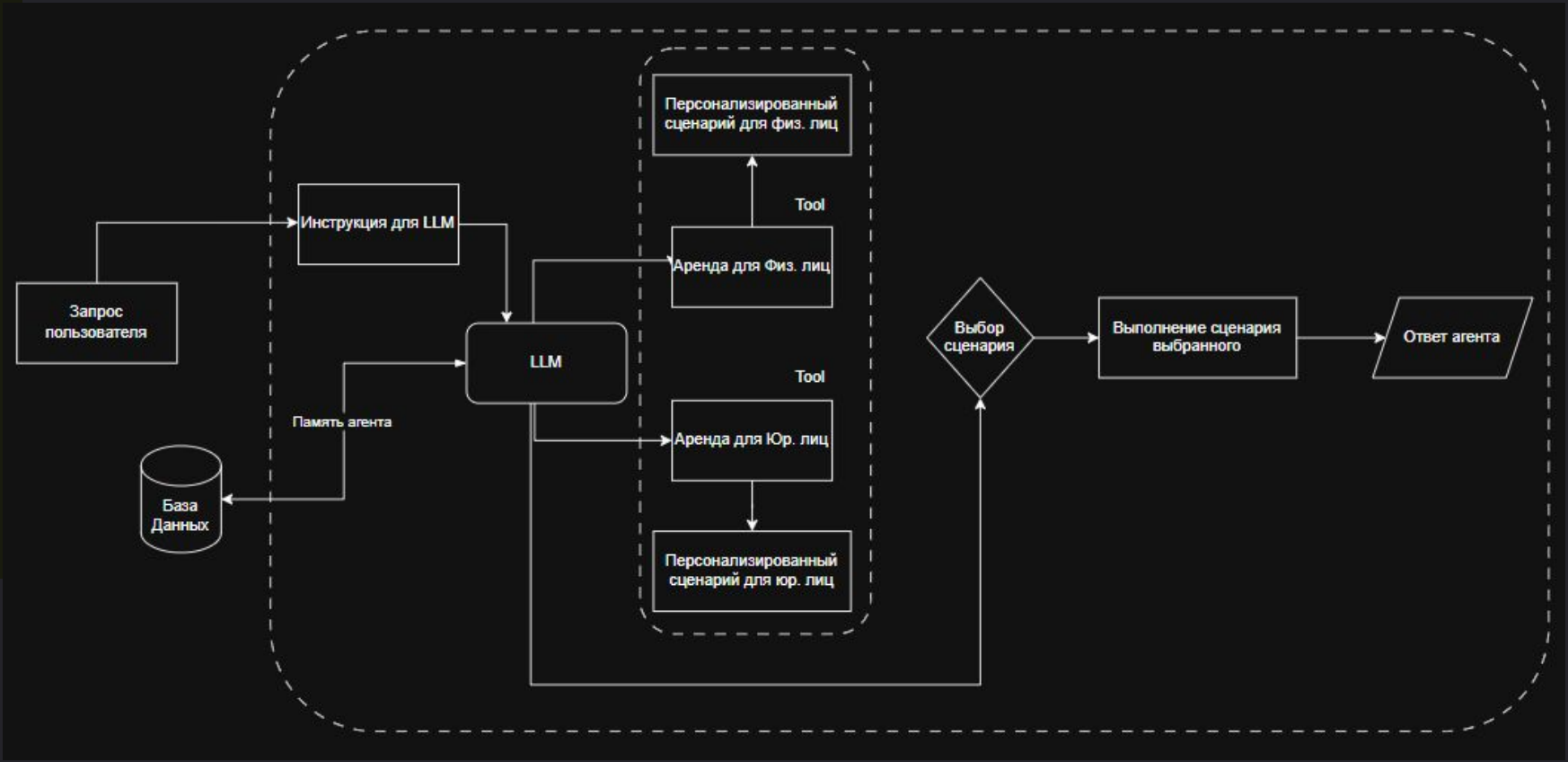
Запрос: “Построй прогноз по медиапотреблению населения на следующий квартал.”
Что делает агент:
1. Определяет подходящий инструмент — например, подключается к базе аналитики медиахолдинга или API соцсетей
2. Загружает данные за предыдущие периоды, очищает и агрегирует их.
3. Формирует понятный отчёт с прогнозом и визуализацией.

**Ответ агента:**
“Согласно данным за последние 6 месяцев, прогноз медиапотребления на следующий квартал:
• Рост онлайн-видео на 12%;
• Снижение интереса к печатным СМИ на 8%;•

# Сравнение агента с языковой моделью

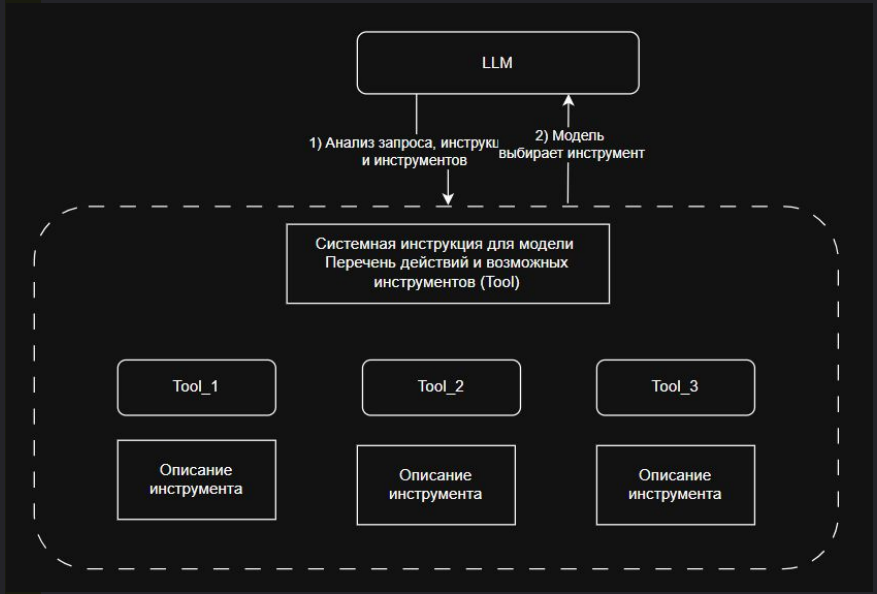


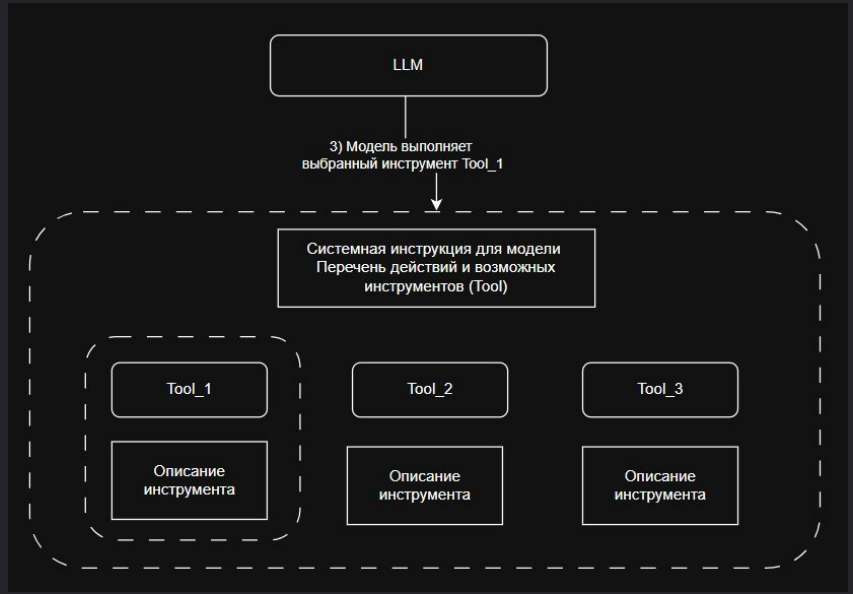
# Пример агента



# Место LLM в агентной системе

С LLM можно работать как по API, так и запускать ее локально, при построении агентной системы




Важно! Работа модели происходит в два этапа: 1) Выбор инструмента, 2) Выполнение Инструмента
Выполнение инструмента может проводить другая модель (например, слабая модель - для экономии ресурсов)

# Схема агента

**Схема простого агента от Langchain**

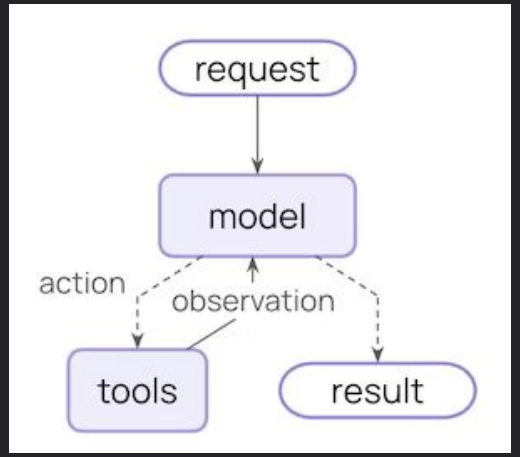
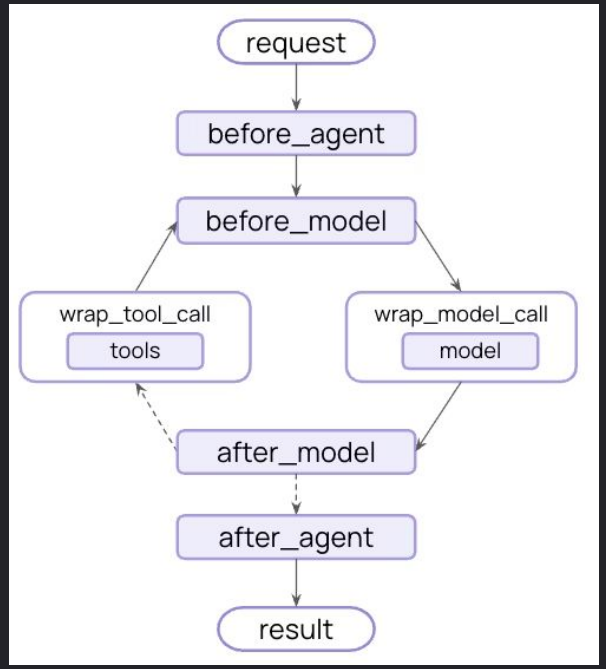
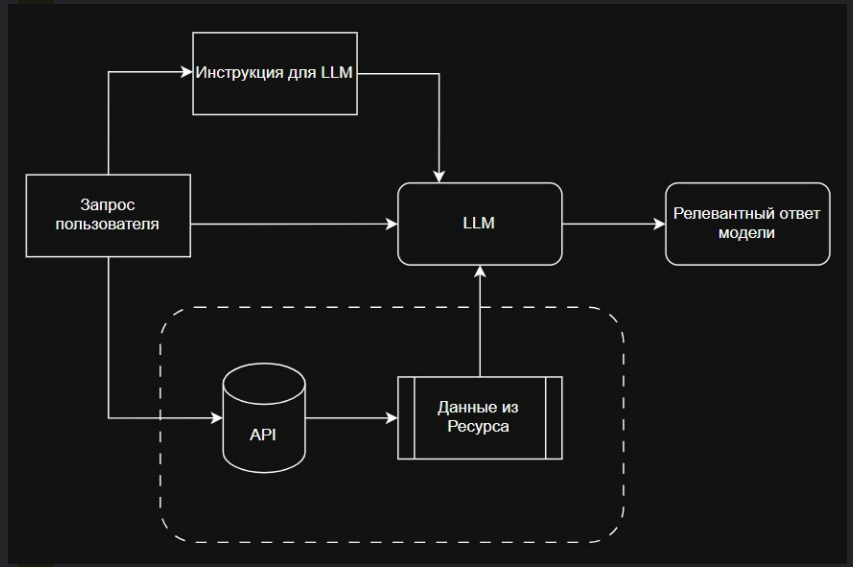



**Схема продвинутого агента от Langchain**

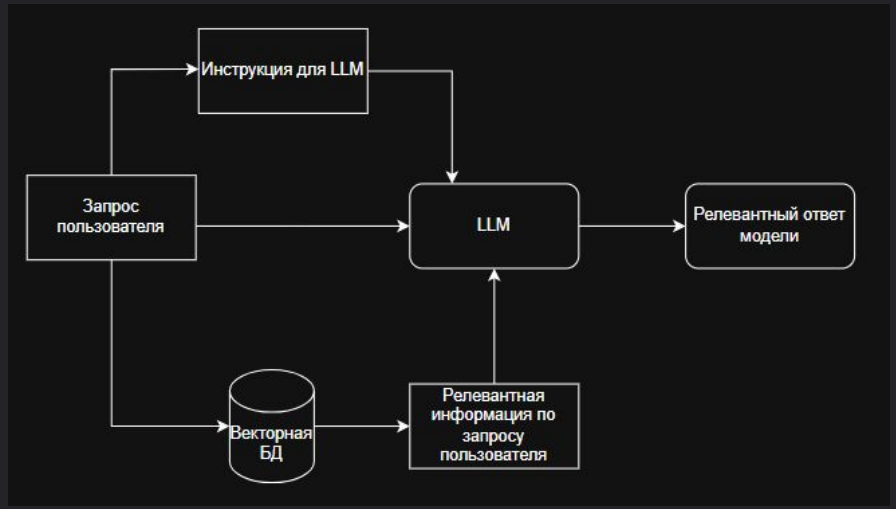


# Обогащение знаний

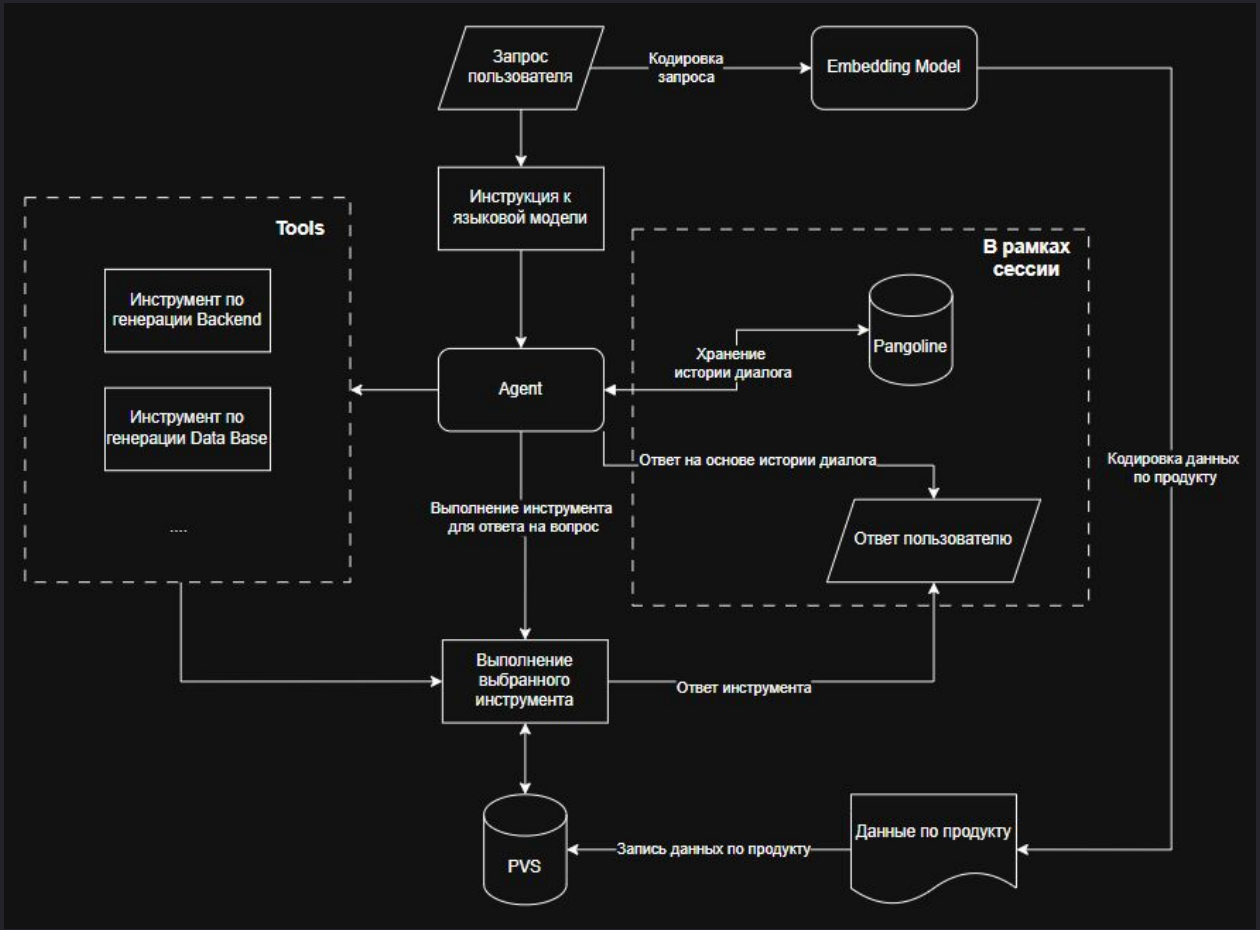
## Внешние API




## Векторная база знаний (RAG)



# Что мы делаем на работе



# Работа с языковой моделью, как частью агента

**Как можно улучшить ответ LLM?**

**Техники промптинга:**

- Написать четкий системный промпт
- Задать несколько примеров ответов
- Указать четкую структуру вывода ответа
- Указать целевую аудиторию
- Указать ограничения и требования

**Дополнительное улучшение ответов языковой модели:**

1. RAG поиск – поиск информации на основе ваших данных
2. Поиск в интернете (используются сторонние API)
3. Подключение памяти диалога

**Быстрый старт - Langflow , OpenAI Assistants, AI Studio**

# Составление промпта

| Тип промпта | Описание | Примеры использования |
|---|---|---|
| **Системный (system prompt)** | Определяет **поведение модели** и её “личность”. Задаёт контекст, стиль ответов, правила, ограничения. | «Ты — эксперт по финансовым рынкам, отвечай кратко и точно», «Действуй как помощник для изучения Python». |
| **Пользовательский (user prompt)** | **Запрос пользователя**. Модель реагирует на него, опираясь на системный промпт. | «Объясни, что такое облигации простыми словами», «Напиши пример кода на Python для сортировки списка». |

## Элементы промпта

1) **Задача** — конкретная задача или инструкция, которую вы хотите, чтобы модель выполнила.
2) **Контекст** — внешняя информация или дополнительный контекст, который может направить модель на более точные ответы.

3) **Входные данные** — вход или вопрос, на который мы хотим найти ответ.

4) **Индикатор вывода** — тип или формат вывода.

# Составление промпта

**Пример -** Ассистент по генерации тестовых заданий по тексту документа.

## Системный промпт №1

Ты выступаешь в роли методиста в сфере финансов. Сгенерируй 5 тестовых задания средней сложности по финансовой тематике.

• Каждое задание должно быть уникальным и охватывать различные аспекты финансов (например, финансовый анализ, бухгалтерский учет, налогообложение, инвестиции, финансовое планирование).
• Формулируй вопросы чётко и понятно.
• Не предоставляй ответы — только задания.

# Вывод
Список из 5 тестовых заданий, каждое отдельным пунктом, формулировка каждого задания не менее одного предложения.

## Системный промпт №2

Ты выступаешь в роли методиста в сфере финансов. Твоя задача — сгенерировать 5 тестовых заданий средней сложности по финансовой тематике для студентов 2 курса направления "Финансы".

Ты должен создать:
- 3 задания закрытого типа (выбор одного правильного варианта ответа из предложенных),
- 2 задания открытого типа (свободный развернутый ответ).

Используй следующий формат для вывода:
Для заданий закрытого типа:
1. [Формулировка вопроса]
а) [Вариант ответа]
б) [Вариант ответа]
в) [Вариант ответа]
г) [Вариант ответа]
верный ответ — [буква правильного варианта]

Для заданий открытого типа:
открытый вопрос — [Формулировка вопроса]
ответ — [Обоснованный развернутый пример/основные аспекты ожидаемого ответа]

# Составление промпта

**Важно добавлять примеры:**

**Пример задания закрытого типа**

**1.** Какой из следующих отчетов отражает финансовое положение компании на определённую дату?

**а)** Отчет о движении денежных средств
**б)** Отчет о финансовых результатах
**в)** Бухгалтерский баланс
**г)** Отчет о собственном капитале

Верный ответ — в

**Пример задания открытого типа:**

**Открытый вопрос** — Объясните, зачем компания проводит анализ ликвидности баланса.

**Ответ** — Компания анализирует ликвидность баланса для оценки своей способности своевременно погашать краткосрочные обязательства.

# Советы

**Использовать последние модели**

**Инструкция в начале промпта, контекст в конце. Для выделения контекста использовать ### или “””**

Summarize the text below as a bullet point list of the most important points.

Text: """
{text input here}
""«

**Желаемый формат вывода также передавать на конкретных примерах**

**Указывать, что НУЖНО сделать и избегать формулировок, что нельзя делать**

**Добавлять ключевые слова в конце, например, Итоговый ответ: Скрипт Pyhton import**

**Четкая инструкция без воды, лучше указывать конкретные значения цифрами!**
Use a 3 to 5 sentence paragraph to describe this product.

The description for this product should be fairly short, a few sentences only, and not too much more.

# Влияние температуры на ответ модели

**Температура** — это параметр генерации текста в языковой модели, который регулирует степень случайности выбора следующего токена

| Значение температуры | Характер ответа | Особенности |
|---|---|---|
| **0** | Максимально предсказуемый, «сухой» | Всегда почти одинаковый ответ, без креатива |
| **≈1** | Баланс: связный и разнообразный | Ответы могут отличаться, но остаются логичными |
| **>1** | Творческий, непредсказуемый | Больше вариативности, но риск галлюцинаций |

Температура управляет **вариативностью и креативностью**, но не «умственными способностями» модели.

# Влияние температуры

Температура = 1

User

Объясни, что такое токен в LLM простыми словами.

Assistant

Конечно! В больших языковых моделях (LLM) **токен** — это маленькая часть текста, на которую модель разбивает входные данные, чтобы их понять и обработать.

Проще говоря, вместо того чтобы смотреть на целые слова или предложения, модель смотрит на небольшие кусочки — это могут быть отдельные слова, части слов или даже отдельные знаки препинания. Эти кусочки и называются токенами.

Например, слово "**привет**" может стать одним токеном, а слово "**большой**" модель может разбить на два токена, например "больш" и "ой", если она так обучена.

Таким образом, токены — это строительные блоки текста для языковой модели, с которыми она работает, чтобы понимать и генерировать текст.

Температура = 0.3

User

Объясни, что такое токен в LLM простыми словами.

Assistant

Токен в LLM (большой языковой модели) — это маленький кусочек текста, на который модель разбивает предложение, чтобы понять и обработать его. Например, токеном может быть слово, часть слова или даже отдельный знак препинания. Модель работает с токенами, а не с целыми предложениями, чтобы лучше анализировать и генерировать текст.

# Как работать с галлюцинациями?

Составление промпта, учитывающего «тонкие» места

Необходимо сформировать промпт, который:
* минимизирует галлюцинации,
* даёт понятную структуру ответа (обоснование).

**Добавление ограничений**
`Отвечай только фактами. Если не уверен — напиши "неизвестно".`
**Просьба о пошаговом рассуждении**
`Объясни шаг за шагом, на основании чего даёшь ответ.`
**Ограничение креативности**
Параметры генерации: низкая температура, умеренный Top-p (см. рекомендации ниже).

Готовые шаблоны промптов

Отвечай только фактами. Если не уверен — напиши "неизвестно". Объясни шаг за шагом, на основании чего даёшь ответ.

Отвечай только фактами. Если не можешь подтвердить факт — напиши "неизвестно" и не придумывай.
Структура ответа:
1) Краткий ответ (1–2 предложения).
2) Пошаговое обоснование (как пришёл к выводу).
3) Источники/примечания (если есть).
Если у тебя нет данных — явно укажи, что информации нет.

# Инструменты для уменьшения галлюцинаций

## Инструменты для уменьшения галлюцинаций

**A. RAG (Retrieval-Augmented Generation)**
Модель сначала ищет информацию в базе знаний, потом генерирует ответ.
Пример: LangChain.
Векторные базы данных: Chroma DB, Faiss, Qdrant

**B. Фильтры и постобработка**
Проверка ответов на факты через внешние API (например, WolframAlpha для числовых/научных данных, SerpAPI для поиска во внешних источниках).

**C. Middleware у агентов.**
Позволяет обрабатывать ответ агента и проверять его на валидность

Эффект: модель опирается на реальные данные → меньше вымышленных фактов.

**Полезные ссылки:**
SerpAPI https://serpapi.com
Qdrant https://python.langchain.com/docs/integrations/vectorstores/qdrant
Chroma DB https://python.langchain.com/docs/integrations/vectorstores/chroma

# Фиксированный ответ модели - Response Format

Наилучший вариант задания строгого формата ответа Structured Output

Json Schema

```
response_format={
   "type": "json_schema",
   "json_schema": {
   "name": "answer_schema",
   "schema": {
     "type": "object",
     "properties": {
        "answer": {
             "type": "string",
             "description": "Ответ"}}
```

Pydantic Schema

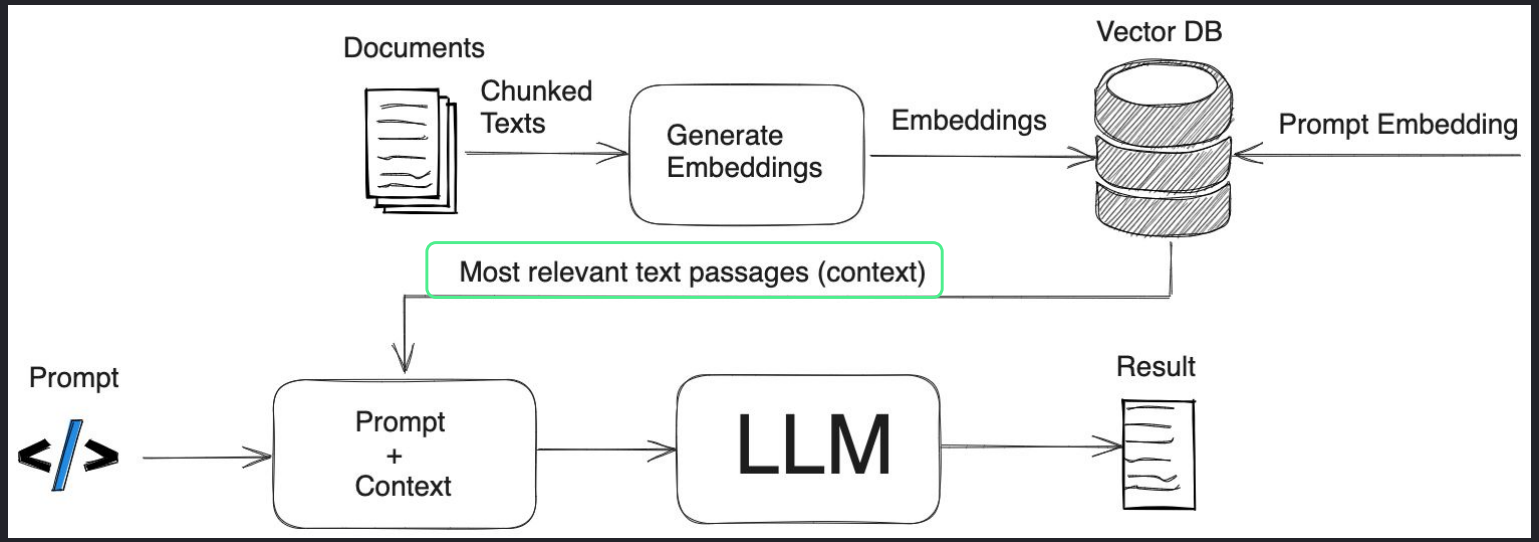
```
response_format={
    "type": "json_schema",
    "json_schema": {
       "name": "transaction_response",
       "schema":
TransactionResponse.model_json_schema(),
       "strict": True  # строгое соответствие
схеме}}
```

```
from pydantic import BaseModel
class TransactionResponse(BaseModel):
    answer: str
```

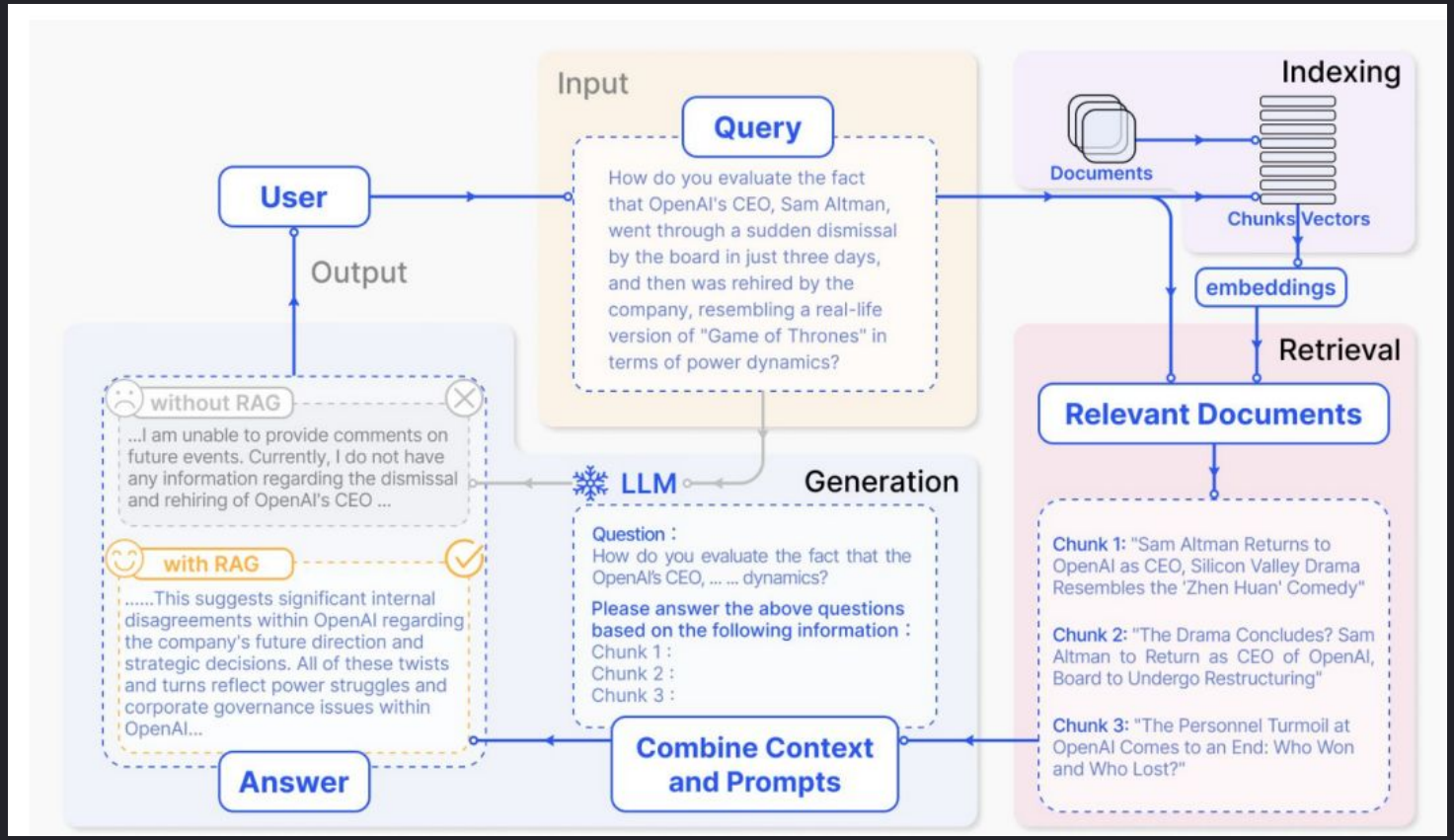
**Плюсы Pydantic Schema**

**Типобезопасность** – проверяет типы сразу при создании объекта.
**Меньше дублирования** – один класс = и схема, и модель данных..
**Читаемость** – код похож на бизнес-логику, а не на JSON-описание.

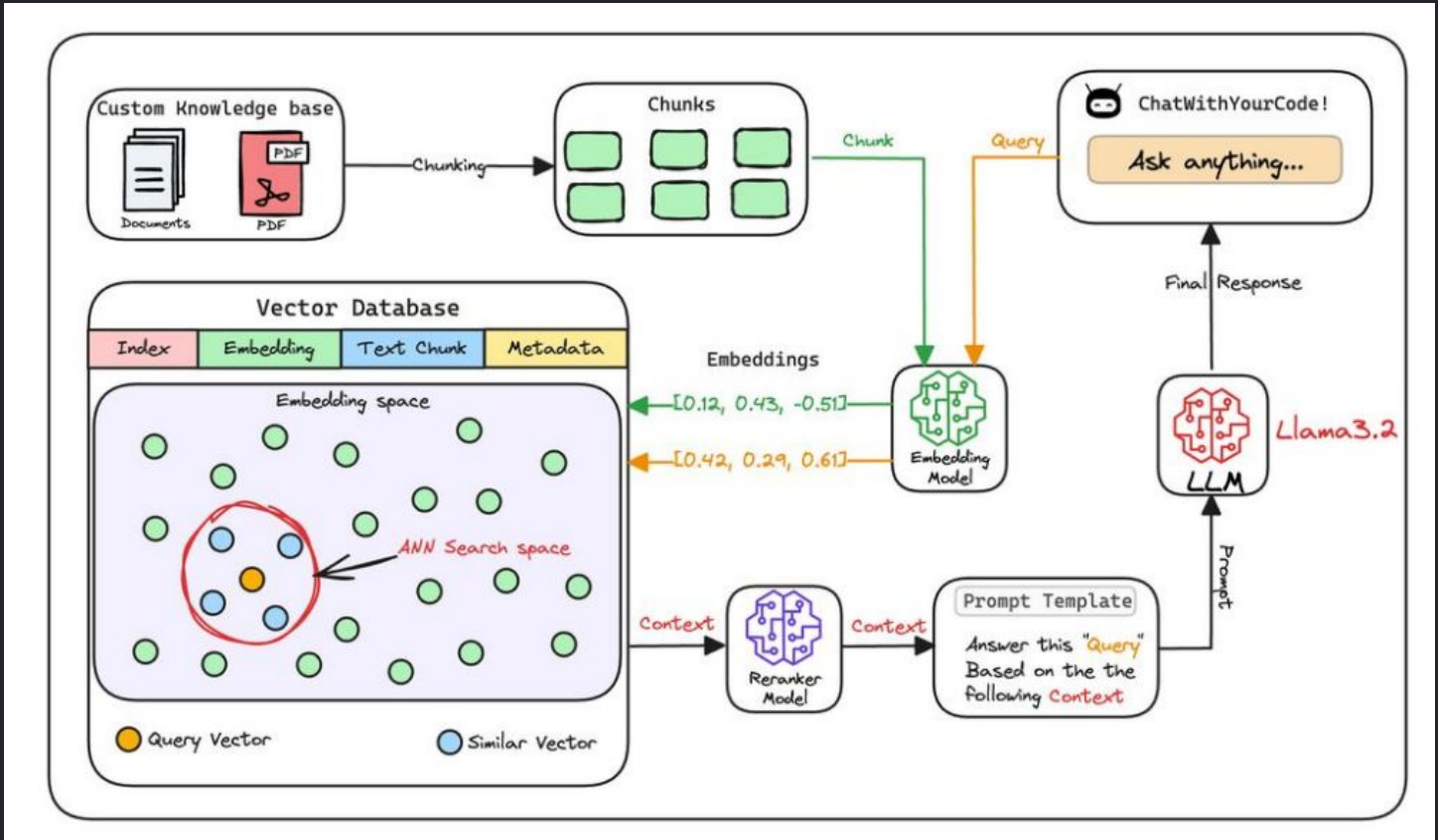
# RAG = извлечение информации из базы данных



# Дополнительно - RAG

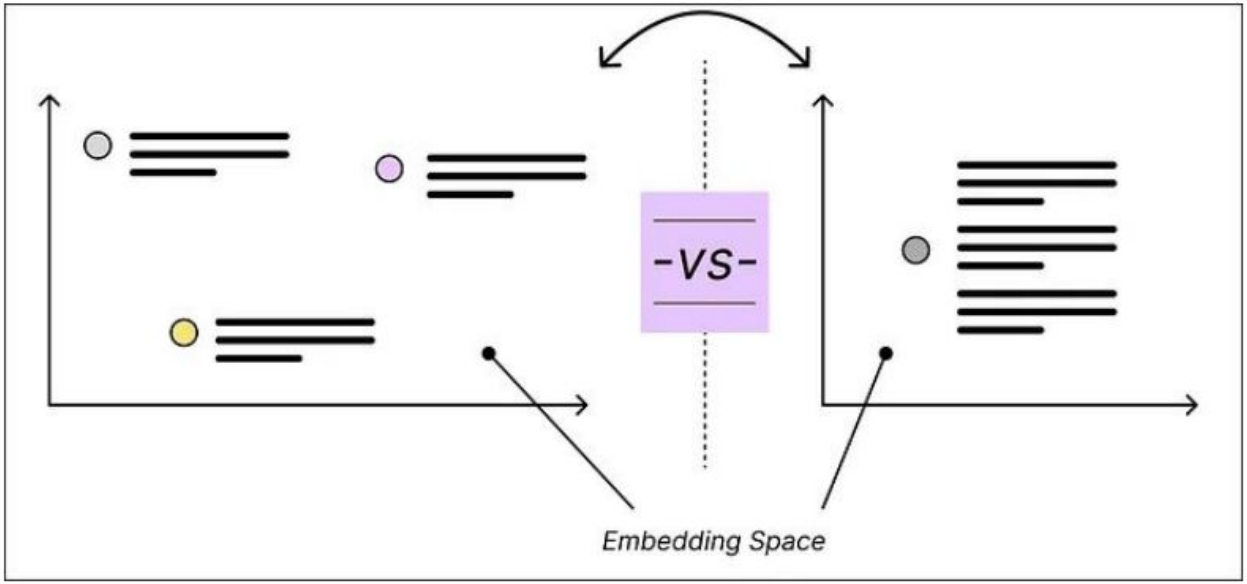


# Дополнительно - RAG

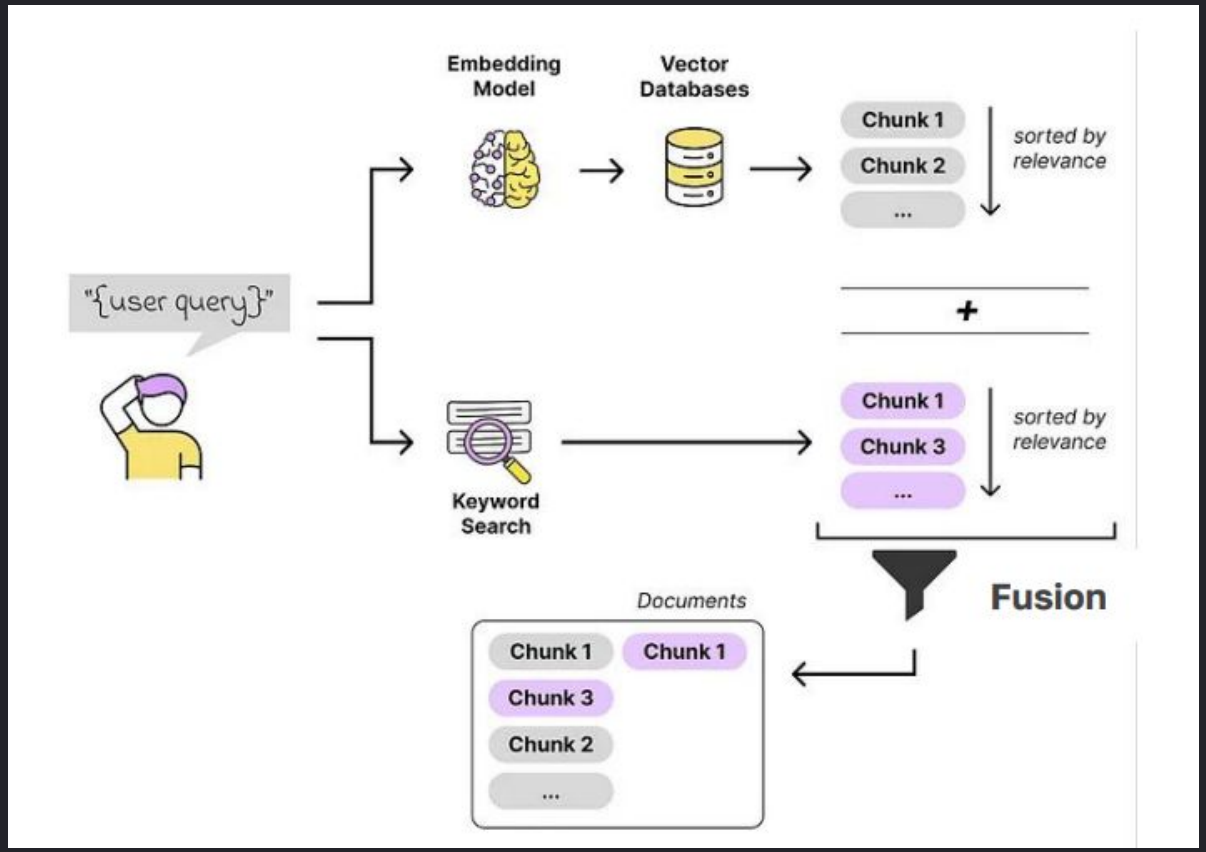


# Дополнительно - RAG

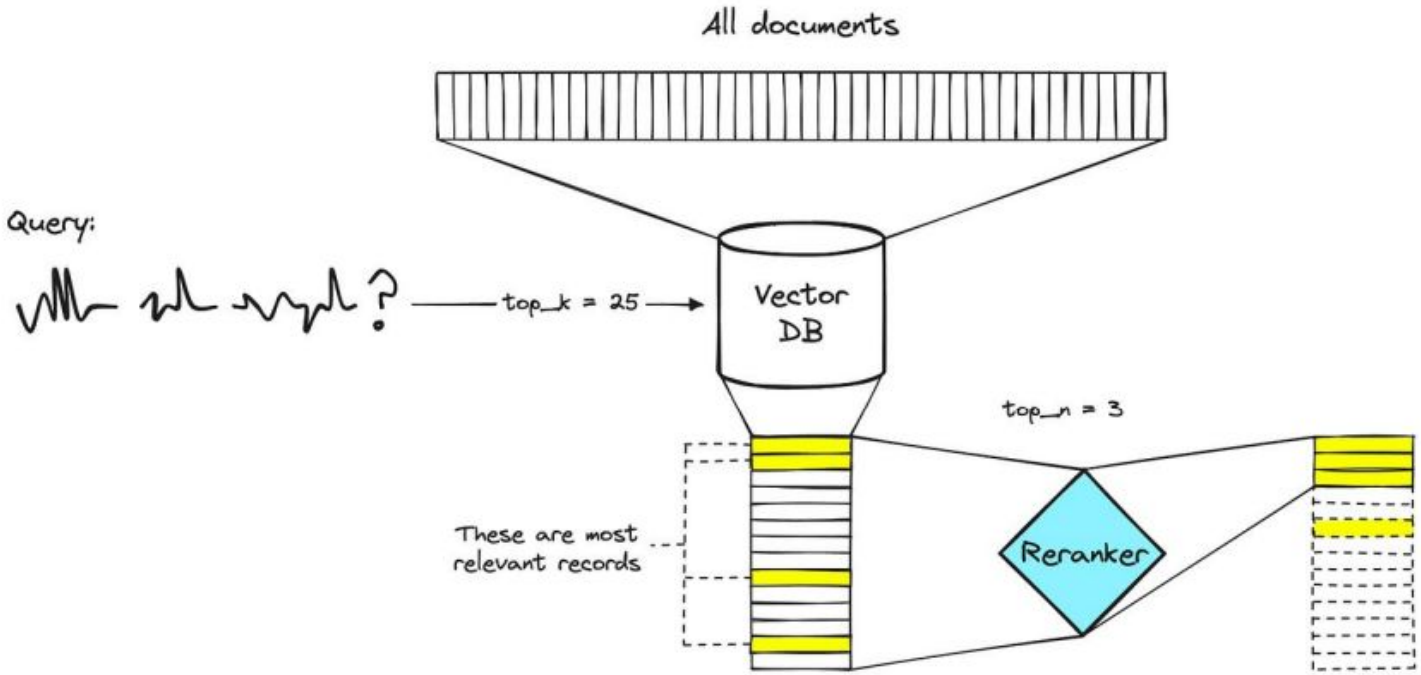
## Pre-Retrieval. Chunk strategy



# Дополнительно - RAG

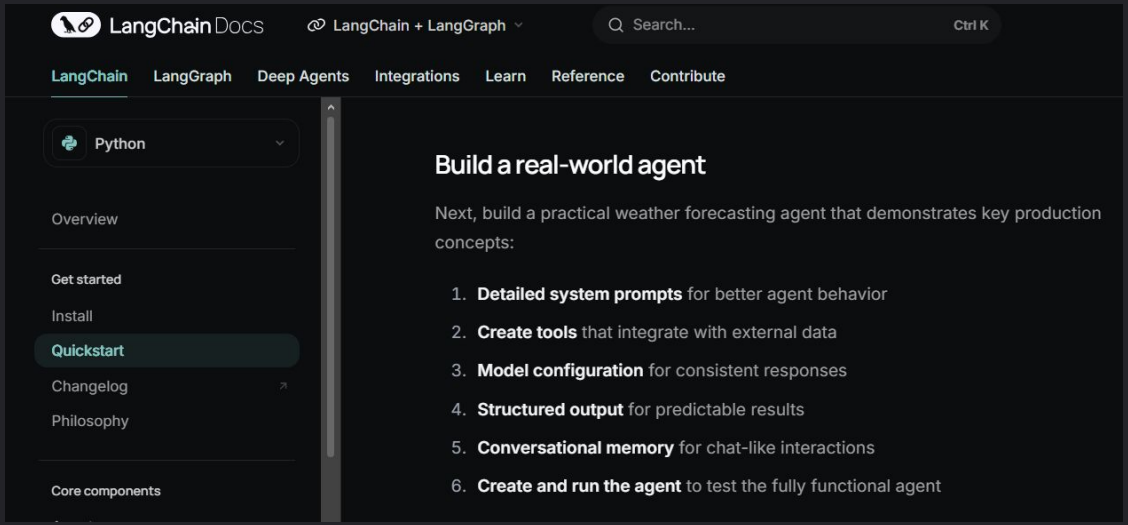


# Дополнительно - RAG




A two-stage retrieval system. The vector DB step will typically include a bi-encoder or sparse embedding model.

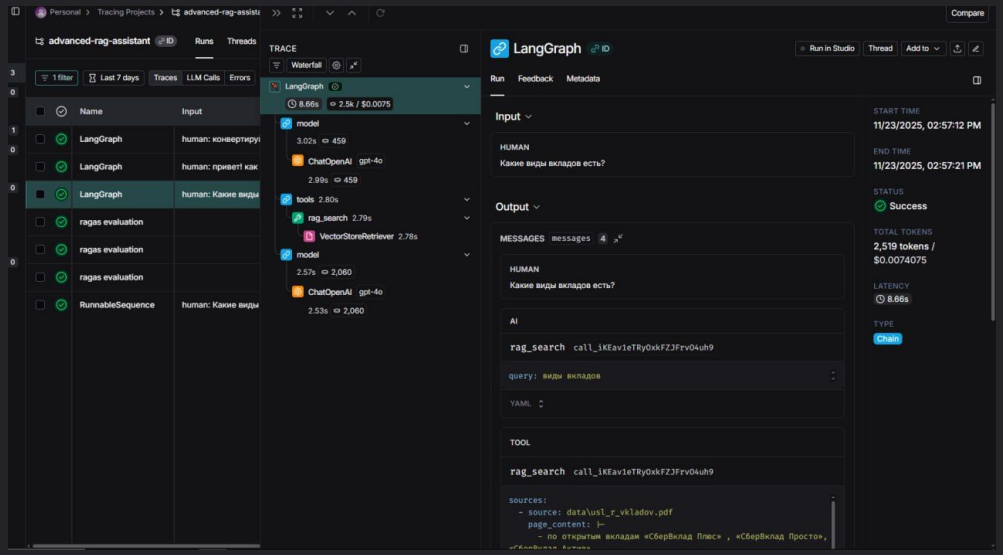
# Библиотека Langchain




**Почему Langchain?**

- Простой функционал по созданию агента - связывает модель, инструменты, память и логику в одном классе
- Легкость подключения к API, БД, инструментам и тп.
- Поддерживает сохранение памяти с различными БД — агенты могут помнить прошлые шаги и действовать в рамках всей истории диалога
- Дополнительный функционал по валидации агентов и их ответов
- Возможность расширенных настроек функций агента с использованием блока Middleware
- Множество примеров использования для быстрого старта

# Библиотека LangGraph




**Почему LangGraph?**

- State Machine / Graph-based execution — агент описывается как граф состояний
- Явные состояния (State) — полный контроль над тем, какие данные передаются между шагами
- Упрощает отладку и контроль — легко видеть, где и почему агент принял решение
- Возможность отследить потребляемые ресурсы языковой моделью: время выполнения, расход токенов, стоимость
- Хранилище промптов с версионированием, особенно хорошо для реальных проектов и тестирования промптов
- Существует аналог LangGraph - LangFuse (ничем ему не уступает)